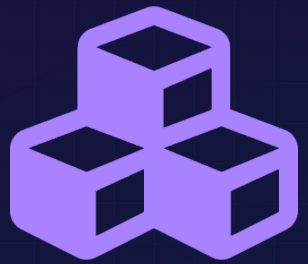
Architecture & Design Patterns

Guiding AI Toward Good Patterns

Week 6 — Phase 2: Engineering



"The goal of good architecture is to defer decisions. With AI, it's to make them explicit."

Instructor: Goker Ezberci

Vibe Coding 101: From Idea to Shipped Product

github.com/goker/vibe-coding-101-for-software-engineers

Today's Agenda

01 **The Spaghetti Problem**
Why AI puts everything in one file

02 **Three Architecture Patterns**
Repository, Service Layer, Middleware

03 **Instruction Files as Guards**
CLAUDE.md enforces patterns at generation time

04 **The Dependency Rule**
Dependencies flow inward—Robert C. Martin

05 **The Refactoring Workflow**
Generate → Identify → Refactor → Verify

06 **Lab: Refactor Your Project**
Apply clean architecture to your Week 3 code

The Spaghetti Problem

AI Default: Everything in One File

```
// index.js — 500+ lines
const express = require('express');
const db = new sqlite3.Database();

// Routes + Auth + Validation + DB + Business Logic
app.post('/events', (req, res) => {
  // Auth check
  // Validation
  // DB queries
  // Business logic
  // Return response
});
// ... 100 more routes like this
```

Problems

- ✗ Impossible to test
- ✗ No code reuse
- ✗ Changes cascade
- ✗ Can't reason about it
- ✗ Hard to onboard

The Clean Way: Modular Structure

routes/

HTTP handlers only

services/

Business logic

repositories/

Data access

middleware/

Auth, validation

Result: Each file does ONE thing. Tests pass. Code grows without chaos.

Three Patterns That Work With AI

Repository Pattern

All data access in one place

✓ Isolates database queries

✓ Easy to mock for tests

✓ One place to change DB logic

Service Layer

Business logic separated from routes

✓ Routes → HTTP only

✓ Services → logic

✓ Testable without server

Middleware Pattern

Cross-cutting concerns reused

✓ Auth applied consistently

✓ Validation in one place

✓ Error handling centralized

Instruction Files as Architecture Guards

Use CLAUDE.md to enforce patterns at generation time, not refactoring time

```
# CLAUDE.md

## Mandatory Rules

### Rule 1: No DB in Routes
- db.run/get/all ONLY in repositories/
- Routes delegate to services

### Rule 2: Business Logic in Services
- Validation, rules, coordination
- Never in route handlers

### Rule 3: Reusable Middleware
- Auth in middleware/auth.js
```

Before (Without CLAUDE.md)

```
// Routes + DB + Logic
app.post('/events', (req, res) => {
  db.run('INSERT...'); // Wrong!
});
```

After (With CLAUDE.md)

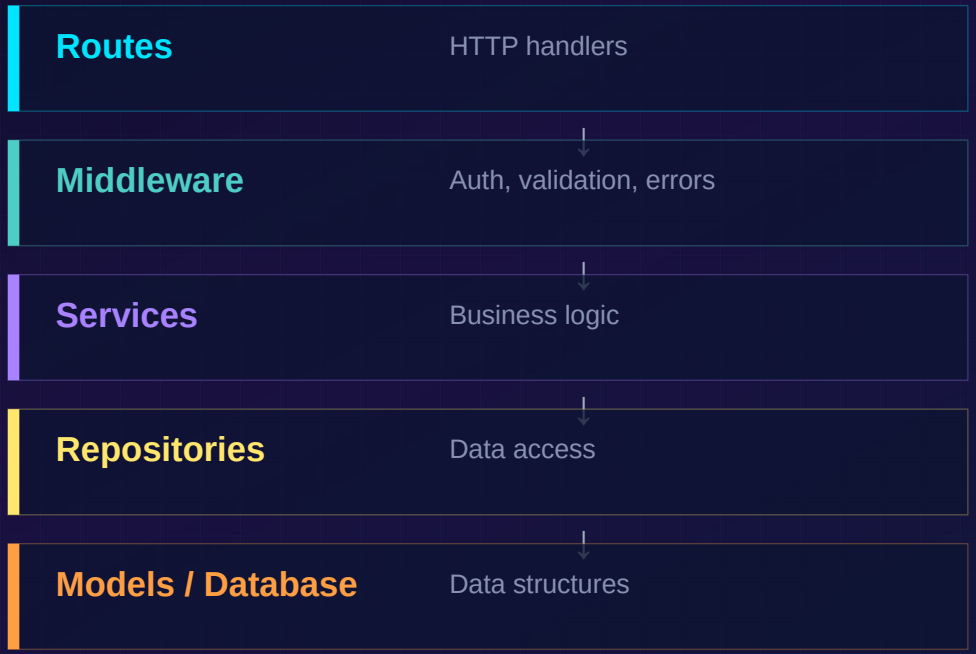
```
// Route delegates to service
router.post('/', async (req, res, next) => {
  eventService.create(...) // Right!
});
```

CLAUDE.md is a contract. Show AI the pattern once, and it follows it consistently across new code.

No more code review comments: 'Please move this to a service.' Just point to the file.

The Dependency Rule

Robert C. Martin: Source dependencies flow inward. No circular dependencies.



Key Rules

✓ Routes → Services (outer → inner)

✓ Services → Repositories (outer → inner)

✗ Repositories → Services (no circles!)

The Refactoring Workflow

How to systematically clean up messy AI code

1 GENERATE

Ask AI for working code
(messy is fine)

→

2 IDENTIFY

Scan for violations
(checklist in README)

→

3 REFACTOR

Ask AI to reorganize
(use CLAUDE.md rules)

→

4 VERIFY

Tests pass
Architecture tests pass

Identify Violations Checklist:

- ☐ db.run/get/all in route handlers?
- ☐ Business logic in routes?
- ☐ Auth code duplicated?
- ☐ No error middleware?
- ☐ Can't test service without DB?

References & Further Reading

Architecture & Design

Martin Fowler — Repository Pattern

Robert C. Martin — Clean Architecture

Patterns of Enterprise App Architecture

Domain-Driven Design (Eric Evans)

AI Coding Patterns

Anthropic — Prompt Engineering Best Practices

Anthropic — Extending Claude with Tools

CodeScene — Agentic AI Coding Patterns

Cursor — .cursorrules Documentation

Instruction Files & Context

Anthropic — Claude.md and Project Context

Gemini — System Instructions Guide

Cursor — Rules Configuration

Testing & Validation

Martin Fowler — Test Pyramid

pytest Documentation

Vitest — Fast Unit Testing Framework

Lab Exercise: Refactor Your Project

Part A

Refactor One Route

- ☐ 1. Pick one route
- ☐ 2. Create Repository
- ☐ 3. Create Service
- ☐ 4. Update Route
- ☐ 5. Tests pass
- ☐ 6. Push to GitHub

Part B

Build CLAUDE.md

- ☐ 1. Create CLAUDE.md
- ☐ 2. List 5+ rules
- ☐ 3. Show examples
- ☐ 4. Include file structure
- ☐ 5. Test with AI
- ☐ 6. Iterate rules

Part C

Architecture Tests

- ☐ 1. Create test file
- ☐ 2. Check: no DB in routes
- ☐ 3. Check: folders exist
- ☐ 4. Check: no circular deps
- ☐ 5. Commit with CI passing
- ☐ 6. Document in README

Submission

Push to GitHub: refactored code • CLAUDE.md • architecture tests passing • Updated README

Commit message should explain what was refactored and why it follows the Dependency Rule.

Key Takeaways

1. AI defaults to spaghetti code. Use patterns (Repository, Service, Middleware) to guide it
2. CLAUDE.md is a contract. Write rules once, AI follows them in all new code
3. The Dependency Rule: dependencies flow inward. No circular dependencies
4. Refactor in four steps: Generate → Identify → Refactor → Verify
5. Architecture tests validate structure. They're as important as feature tests

Course Materials: github.com/goker/vibe-coding-101-for-software-engineers